

Diplomarbeit

EduShelf

Verfasser: Elias Welt 5AHITS
Phillip Rubak 5AHITS

Betreuer: Dipl.-Ing. Gerald Stoll

Schuljahr: 2025/26

Abgabevermerk:

Datum: 07.04.2025

übernommen von:

Höhere Technische Bundeslehranstalt Hollabrunn

Höhere Lehranstalt für Informationstechnologie

EIDESSTATTLICHE ERKLÄRUNG

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe.

Hollabrunn, am 07.04.2025, Elias Welt

Hollabrunn, am 07.04.2025, Phillip Rubak

1 HINWEISE

Die vorliegende Diplomarbeit wurde in Zusammenarbeit mit der Firma "X-Works" ausgeführt.

Die in dieser Diplomarbeit entwickelten Prototypen und Software-Produkte dürfen ganz oder auch in Teilen von Privatpersonen oder Firmen nur dann in Verkehr gebracht werden, wenn sie diese selbst geprüft und für den vorgesehenen Verwendungszweck für geeignet befunden haben.

Es wird keinerlei Haftung übernommen für irgendwelche Schäden, die aus der Nutzung der hier entwickelten oder beschriebenen Bestandteile des Projekts resultieren.

Für alle Entwicklungen gilt die GNU General Public License [<http://www.gnu.org/licenses/gpl.html>] der Free Software Foundation, Boston, USA in der Version 3.

2 SCHLÜSSELBEGRIFFE

- TeamTrackr
- Android - Entwicklung
- iOS - Entwicklung
- Backend
- Frontend
- AI

3 DANKSAGUNGEN

Wir möchten an dieser Stelle unseren aufrichtigen Dank all jenen aussprechen, die uns während der Entstehung dieser Diplomarbeit unterstützt haben.

Unser besonderer Dank gilt Tailored Apps, unserer Partnerfirma, die uns nicht nur mit wertvollen Einblicken und praxisnaher Unterstützung begleitet hat, sondern auch eine entscheidende Rolle in der erfolgreichen Umsetzung unseres Projekts gespielt hat. Die Zusammenarbeit mit euch war für uns eine wertvolle Erfahrung.

Ebenso möchten wir uns herzlich bei Herrn Geischläger, unserem Diplomarbeitsbetreuer, bedanken. Seine fachliche Expertise, seine konstruktiven Anregungen und seine stetige Unterstützung haben maßgeblich zur Qualität unserer Arbeit beigetragen.

Ein großer Dank gilt auch allen, die uns auf diesem Weg unterstützt haben – sei es durch hilfreiches Feedback, motivierende Worte oder ihr Verständnis und ihre Geduld.

Ohne diese wertvolle Unterstützung wäre unsere Diplomarbeit in dieser Form nicht möglich gewesen.

DIPLOMARBEIT DOKUMENTATION

Name der Verfasser/innen	Elias Welt, Phillip Rubak
Jahrgang Schuljahr	2025/26
Thema der Diplomarbeit	EduShelf
Kooperationspartner	—

Aufgabenstellung	Im Rahmen dieser Diplomarbeit wird 'EduShelf' entwickelt, eine webbasierte Plattform zur Verwaltung und Interaktion mit Bildungsunterlagen. Ziel ist es, Schülern und Lehrern eine zentrale Lösung zu bieten, um Dokumente zu organisieren und KI für eine effiziente Informationsbeschaffung zu nutzen. Die Anwendung verbindet Dokumentenmanagement mit einem KI-gestützten Chat-System, das es Benutzern ermöglicht, Fragen zu ihren hochgeladenen Inhalten zu stellen (RAG - Retrieval Augmented Generation). Ein zentraler Aspekt ist die nahtlose Integration eines modernen Web-Frontends mit einem leistungsfähigen Backend und einem KI-Server. Das Backend steuert die Datenverarbeitung und -speicherung, während der KI-Server die Intelligenz für die Chat-Funktionalität bereitstellt. Diese Plattform soll den Zugriff auf und die Nutzung von Bildungsressourcen modernisieren.
------------------	--

Realisierung	Das Projekt umfasst die Entwicklung einer responsiven Webanwendung und eines leistungsfähigen Backend-Systems. Das Frontend wird mit React umgesetzt und bietet eine moderne Benutzeroberfläche. Das Backend, implementiert in .NET, steuert die Datenspeicherung und -verarbeitung. Ein dedizierter KI-Server, basierend auf Python und Ollama, übernimmt die LLM-Operationen für die Chat-Funktionalität. Das System nutzt Containerisierung (Docker) für einfache Bereitstellung und Skalierbarkeit.
--------------	---

Ergebnisse	Das Ergebnis ist die voll funktionsfähige Plattform 'EduShelf'. Zu den Kernfunktionen gehören Benutzerauthentifizierung, Upload und Indizierung von Dokumenten sowie ein interaktiver KI-Chat. Nutzer können Accounts erstellen, ihre Dokumentenbibliothek verwalten und durch die KI-Schnittstelle sofortige Antworten aus ihren Lernunterlagen erhalten. Das System demonstriert die effektive Integration moderner Webtechnologien und künstlicher Intelligenz im Bildungsbereich.
------------	---

Typische Grafik, Foto etc. (mit Erläuterung)	MISSING SVG: img/blockschaltbildSVG.svg
---	--

Teilnahme an Wettbewerben, Auszeichnungen	
--	--

Möglichkeiten der Einsichtnahme in die Arbeit	HTL Hollabrunn Anton Ehrenfriedstraße 10 2020 Hollabrunn
--	--

Approbation (Datum / Unterschrift)	Prüfer/Prüferin	Direktor/Direktorin Abteilungsvorstand/ Abteilungsvorständin
---------------------------------------	-----------------	--

DIPLOMA THESIS

Documentation

Author(s)	Elias Welt, Phillip Rubak,
From Academic year	2025/26
Topic	EduShelf
Co-operation partners	TailoredApps

Task Description	This diploma project focuses on developing 'EduShelf', a web-based platform for managing and interacting with educational content. The goal is to provide students and teachers with a centralized solution for organizing documents and leveraging AI for efficient information retrieval. The app integrates document management with an AI-powered chat system that allows users to ask questions about their uploaded content (RAG - Retrieval Augmented Generation). A key aspect is the seamless integration of a modern web frontend with a powerful backend and an AI server. The backend handles data processing and storage, while the AI server provides the intelligence for the chat functionality. This platform aims to modernize how educational resources are accessed and utilized.
------------------	---

Implementation	The project involves developing a responsive web application and a robust backend system. The frontend is built with React, ensuring a modern and user-friendly interface. The backend, implemented in .NET, manages data storage and processing. A dedicated AI server, utilizing Python and Ollama, handles the large language model operations for the chat functionality. The system uses containerization (Docker) for easy deployment and scalability.
----------------	---

Results	The expected outcome is a fully functional 'EduShelf' platform. Key features include user authentication, document upload and indexing, and an interactive AI chat. Users can create accounts, manage their document libraries, and obtain instant answers from their study materials through the AI interface. The system demonstrates the effective integration of modern web technologies and artificial intelligence in an educational context.
---------	---

Typische Grafik, Foto etc. (mit Erläuterung)	MISSING SVG: img/blockschaltbildSVG.svg
---	--

Participation in competitions Awards	
---	--

Accessibility of final project thesis	HTL Hollabrunn Anton Ehrenfriedstraße 10 2020 Hollabrunn
--	--

Approval (Date / Signature)	Examiner/s	Head of Department / College

Thema

EduShelf

Verlauf

- 13.06.2024: Das Thema 'TeamTrackr' wurde durch Franz Geischläger angelegt.
- 13.06.2024: Sie wurden zum Thema hinzugefügt.
- 13.06.2024: Sie wurden zum Thema hinzugefügt.
- 13.06.2024: Sie wurden zum Thema hinzugefügt.
- 16.09.2024: Thema wurde eingereicht.
- 16.09.2024: Thema wurde von Franz Geischläger akzeptiert.
- 16.09.2024: Thema wurde von Wilfried Trollmann akzeptiert.
- 23.09.2024: Thema wurde von Wolfgang Bodei akzeptiert.

Schulische Informationen

- **Schulart:** Technische, gewerbliche und kunstgewerbliche Schulen
- **Klasse:** 5AHITS
- **Abteilung:** IT
- **Schuljahr der abschließenden Prüfung:** 2024/25

Betreuung

Rolle	Name
Hauptverantwortlicher Betreuer	Franz Geischläger (Akzeptiert)
Direktion	Wolfgang Bodei (Akzeptiert)

Kooperationspartner / Auftraggeber

- **Name:** Tailored Apps
- **Adresse:** Heiligenstädterstraße 31/1, 1190 Wien
- **URL:** www.tailored-apps.com
- **E-Mail:** office@tailored-apps.com
- **Ansprechpartner:** Martin Zehetner
- **Telefon:** +43 18902845
- **Typ:** Sonstige

Beteiligte SchülerInnen

Schüler/in	Individuelle Themenstellung	Abteilung
Elias Welt	Frontend, Frontend/Backend Schnittstelle, KI Unterstützung	IT
Phillip Rubak	Backend, KI	IT

Ausgangslage

Derzeit mangelt es an einer zentralisierten Plattform, die es ermöglicht, Bildungsunterlagen effizient zu verwalten und interaktiv mit diesen zu arbeiten. Studierende und Lehrende haben oft Schwierigkeiten, spezifische Informationen in großen Mengen von Dokumenten schnell zu finden. Es fehlt eine integrierte Lösung, die Dokumentenmanagement mit moderner KI-gestützter Analyse verbindet.

Die Anliegen der Themenstellungen umfassen die Entwicklung einer Webplattform sowie eines leistungsfähigen Backends inklusive AI-Server-Anbindung. Die Webapplikation soll folgende Kernfeatures beinhalten:

- Dokumentenmanagement (Upload, Organisation)
- Interaktiver Chat mit Dokumenten (RAG - Retrieval Augmented Generation)

- Benutzerfreundliches Interface für effizientes Lernen

Das Backend soll die Dokumentenverarbeitung steuern, die KI-Modelle anbinden und die Datenhaltung übernehmen.

Zielsetzung

EduShelf bietet eine intuitive Plattform zur Verwaltung von Bildungsressourcen. Durch die Integration von KI können Nutzer direkt mit ihren Unterlagen kommunizieren, Fragen stellen und Zusammenfassungen erhalten. Dies fördert ein tieferes Verständnis der Lerninhalte und spart Zeit bei der Informationssuche.

Das geplante Ergebnis ist eine voll funktionsfähige Webplattform "EduShelf". Die Anwendung ermöglicht:

- Upload und Indizierung von PDF- und Textdokumenten
- KI-basierte Beantwortung von Fragen zu den Inhalten
- Verwaltung der hochgeladenen Dokumente

Das System besteht aus einem Frontend (React), einem Backend (.NET) und einem KI-Server (Python/Ollama), die nahtlos zusammenarbeiten.

Meilensteine

Meilenstein	Datum
Fertigstellung Grundkonzept	20.06.2024
Fertigstellung Backend	22.11.2024
Fertigstellung Frontend	20.12.2024
Fertigstellung Applikationen	24.01.2025
Fertigstellung Dokumentation	14.03.2025

Erklärung

Die unterfertigten Kandidaten/Kandidatinnen haben gemäß den geltenden schulrechtlichen Bestimmungen die Ausarbeitung einer abschließenden Arbeit (Diplomarbeit bzw. Abschlussarbeit) mit folgender Aufgabenstellung gewählt:

TeamTrackr

Individuelle Aufgabenstellungen im Rahmen des Gesamtprojektes:

- Lorenz Geyser (4AHITS): **Backend-Development**
- Suad Demiri (4AHITS): **iOS-Development**
- Kurt Broneder (4AHITS): **Web-Development**
- Stefan Bayer (4AHITS): **Android-Development**

Die Kandidaten/Kandidatinnen nehmen zur Kenntnis, dass die abschließende Arbeit in eigenständiger Weise und außerhalb des Unterrichtes zu bearbeiten und anzufertigen ist, wobei Ergebnisse des Unterrichtes mit einbezogen werden können, die jedenfalls als solche entsprechend kenntlich zu machen sind.

Die Abgabe der vollständigen abschließenden Arbeit hat in digitaler und in zweifach ausgedruckter Form bis spätestens **07.04.2025** beim zuständigen Betreuer/der zuständigen Betreuerin zu erfolgen.

Die Kandidaten/Kandidatinnen nehmen auch zur Kenntnis, dass ein Abbruch der abschließenden Arbeit nicht möglich ist.

Kandidaten/Kandidatinnen:

**Datum und Unterschrift bzw.
Handysignatur:**

Lorenz Geyser (4AHITS)

Lorenz Geyser 13.9.2025

Suad Demiri (4AHITS)

Suad Demiri 13.9.2025

Kurt Broneder (4AHITS)

Kurt Broneder 13.9.2025

Stefan Bayer (4AHITS)

Stefan Bayer 13.9.2025

Inhaltsverzeichnis

1	HINWEISE	5
2	SCHLÜSSELBEGRIFFE	6
3	DANKSAGUNGEN	7
3.1	Motivation für die Arbeit	22
3.2	Umfeld und Stand der Technik	22
3.3	Aufgabenstellung	22
3.3.1	Ziele	23
4	Frontend Implementierung	25
4.1	Einführung und Zielsetzung	25
4.2	Technologie-Stack	26
4.2.1	React (v19)	26
4.2.2	Vite	26
4.2.3	React Router (v7)	26
4.3	Projektsetup und Konfiguration	27
4.4	Softwarearchitektur und Datenfluss	28
4.4.1	Unidirektionaler Datenfluss	28
4.4.2	Routing-Strategie	28
4.5	Datenschicht und API-Integration	29
4.5.1	API-Wrapper	29
4.6	Implementierung der Kernkomponenten	30
4.6.1	Authentifizierungssystem	30
4.6.2	Dateimanagement (Files.jsx)	30
4.6.3	Interaktives Lernen: Das Quiz-Modul	31

4.6.3.1	Datenstruktur	31
4.6.3.2	Ablauf eines Quiz	31
4.6.4	Flashcards (Lernkarten)	31
4.6.5	Chat und KI-Integration	32
4.6.6	Admin Panel und RBAC	32
4.7	Benutzeroberfläche und Design (UI/UX)	33
4.7.1	Styling-Strategie	33
4.7.2	Dark Mode Implementierung	33
4.8	Detaillierte Analyse ausgewählter Module	34
4.8.1	Quiz-Editor Logik (QuizModal.jsx)	34
4.8.1.1	Zustandsverwaltung	34
4.8.1.2	Dynamische Formular-Manipulation	34
4.8.1.3	Validierung und Speichern	35
4.8.2	Flashcards: 3D-Interaktion und State	35
4.8.3	Admin Panel: Kaskadierende Datenabfragen	35
4.9	Deployment und Betrieb	36
4.9.1	Docker-Containerisierung	36
4.9.2	Nginx Konfiguration	37
4.10	Herausforderungen und Lösungen	37
4.10.1	Synchronisation des UI-Status	37
4.10.2	Performance bei großen Dateilisten	38
4.11	Ausblick und Erweiterungsmöglichkeiten	38
4.12	Fazit	38
5	Backend Implementierung	41
5.1	Systemarchitektur	41
5.1.1	Presentation Layer (Controllers)	41
5.1.2	Business Logic Layer (Services)	42
5.1.3	Data Access Layer (Data)	42
5.1.4	Infrastructure Layer	43
5.2	Technologie-Stack	43
5.3	Projekt-Setup und Konfiguration	44
5.3.1	Programm-Initialisierung (Program.cs)	44

5.3.2	Konfiguration (appsettings.json)	45
5.4	Datenmodelle und Datenbankdesign	46
5.4.1	Kernelemente	46
5.4.2	Entity Relationship Diagram (ERD)	48
5.5	API Design und Controller	48
5.5.1	Ausgewählte API-Endpunkte	49
5.6	Sicherheitskonzepte	49
5.6.1	Authentifizierung & Autorisierung	50
5.6.2	Datenvalidierung	50
5.6.3	Error Handling	50
5.7	Ingestion-Pipeline (Datenverarbeitung)	51
5.7.1	1. Textextraktion (OCR & Parsing)	51
5.7.2	2. Chunking (Intelligente Segmentierung)	51
5.7.3	3. Embedding und Vektorspeicherung	52
5.8	Detaillierte Analyse ausgewählter Module	52
5.8.1	Authentifizierung: Secure Cookies und Hashing	53
5.8.1.1	Passwort-Hashing (BCrypt)	53
5.8.1.2	Cookie-Konstruktion	53
5.8.2	Ingestion-Pipeline: HNSW Indexierung	53
5.9	RAG (Retrieval-Augmented Generation) Workflow	54
5.9.1	Systemübersicht und Datenfluss	55
5.9.2	Schritt 1: Intent-Erkennung (IntentDetectionService)	56
5.9.3	Schritt 2: Semantische Suche (RetrievalService)	56
5.9.3.1	Primäre Suche: Name-basiertes Matching	57
5.9.3.2	Fallback: Semantische Vektorsuche	57
5.9.4	Schritt 3: Prompt-Aufbau (PromptGenerationService)	58
5.9.5	Schritt 4: Orchestrierung und Intent-basiertes Branching (ChatService)	58
5.9.6	Zusammenfassung: Retrieval-Augmented Generation	59
5.10	KI-gestützte Lerninhaltsgenerierung	60
5.10.1	Generierungspipeline (5 Schritte)	60
5.10.2	Flashcard-Generierung (FlashcardService)	61
5.10.3	Quiz-Generierung (QuizService)	61

5.11	Infrastruktur & Deployment	62
5.11.1	Docker-Komposition	62
5.11.1.1	Datenbank (PostgreSQL mit pgvector)	62
5.11.1.2	KI-Inferenz (Ollama)	63
5.11.1.3	Object Storage (MinIO)	63
5.11.2	Dockerfile	64
5.12	Resilienz und Stabilität	64
5.12.1	Retry-Policies & Circuit Breaker	64
5.13	Herausforderungen und Lösungen	65
5.13.1	Blockierende Operationen bei großen Dateien	65
5.13.2	LLM Kontext-Fenster-Limits	66
5.13.3	Datenkonsistenz bei Löschoperationen	66
5.14	Qualitätssicherung und Tests	66
5.14.1	Unit Tests (xUnit)	66
5.14.2	Integration Tests	66
5.15	Fazit	67

3.1 Motivation für die Arbeit

Die effektive Integration von Management-Instrumentarien für Organisationsstrukturen, Projektkoordination und Teamführung stellt ein signifikantes Desiderat in der aktuellen digitalen Infrastruktur dar. Gegenwärtige Analysen dokumentieren eine Fragmentierung der verfügbaren Softwarelösungen, wodurch die Realisierung eines kohärenten und effizienten Organisationsmanagements substantiell limitiert wird. Die Diskrepanz zwischen vorhandenen partikulären Anwendungen und der Anforderung nach einer holistischen Managementplattform manifestiert sich in beträchtlichen Effizienzdefiziten sowie suboptimaler Ressourcenallokation innerhalb organisatorischer Einheiten. Diese Forschungsarbeit adressiert die identifizierte Problematik mittels einer systematischen Konzeption und Implementation einer integrierten Softwarelösung zur Überwindung existierender technologischer Limitationen.

3.2 Umfeld und Stand der Technik

Im Kontext der gegenwärtigen technologischen Entwicklung existieren divergente Einzelsysteme zur Adressierung spezifischer Management-Aspekte. Eine kritische Evaluation der Fachliteratur indiziert signifikante Korrelationen zwischen fragmentierten Management-Infrastrukturen und reduzierten Produktivitätsparametern sowie verminderter Nutzerakzeptanz. Die Extrapolation aktueller Entwicklungstendenzen im Bereich digitaler Management-Systeme lässt eine progressive Konvergenz einzelner Funktionalitäten antizipieren, jedoch fehlt bislang eine vollständig interoperable Plattform mit multimodaler Zugänglichkeit. Existierende Systeme offerieren elaborierte Funktionalitäten im Bereich der Kommunikationsfazilitation, während ein anderes System primär exzelliert im Kontext des Projektmanagements. Eine Integration dieser disparaten Ansätze in eine kohärente Systementität wurde bislang nicht realisiert. Die vorliegende Forschungsarbeit positioniert sich in dieser identifizierten Forschungslücke mit dem Ziel der Entwicklung einer konsolidierten Management-Infrastruktur.

3.3 Aufgabenstellung

Die Forschungsaufgabe umfasst die systematische Konzeption und Implementierung eines multimodalen Anwendungssystems mit nativen Applikationen

für iOS, Android und Desktop-Systeme sowie einer komplementären Web-Applikation, unterstützt durch eine zentralisierte Backend-Infrastruktur. Die primären Forschungs- und Entwicklungsziele umfassen:

- Entwicklung eines hochsicheren Authentifizierungsmechanismus unter Berücksichtigung aktueller kryptographischer Standards und Sicherheitsparadigmen
- Implementation eines semantisch strukturierten Kalendersystems mit integrierter Ressourcenallokationsfunktionalität
- Konzeption und Realisierung eines Kommunikationssystems mit bidirektionaler Informationstransmission sowie Integration eines künstlich-intelligenten Konversationssystems zur Optimierung der Kommunikations-effizienz

Die Backend-Architektur erfordert eine optimierte Datenbankstruktur mit adäquater Skalierbarkeit sowie eine performante API-Implementation zur Gewährleistung effizienter plattformübergreifender Datensynchronisation und -konsistenz.

3.3.1 Ziele

Das primäre wissenschaftliche Ziel dieser Forschungsarbeit konstituiert sich in der Entwicklung und Evaluation einer vollständig funktionalen Management-Plattform mit multimodaler Zugänglichkeit auf diversen Endgeräten (iOS, Android, Desktop, Web) unter Berücksichtigung folgender Forschungsziele:

1. Konzeption und Implementation einer optimierten Systemarchitektur mit effizienten Datenflussstrukturen zwischen Frontend- und Backend-Komponenten unter Anwendung aktueller Software-Engineering-Prinzipien
2. Realisierung einer kognitiv ergonomischen Benutzeroberfläche mit konsistenter visueller und funktionaler Repräsentation über heterogene Plattformen hinweg, basierend auf etablierten Usability-Heuristiken
3. Entwicklung eines kryptographisch robusten Authentifizierungssystems zur Gewährleistung der Datenintegrität und -vertraulichkeit

4. Implementation eines multifunktionalen Team- und Projektmanagementsystems mit präzisen Funktionalitäten zur Budgetierung und temporalen Ressourcendisposition unter Berücksichtigung aktueller Projektmanagement-Methodologien
5. Konzeption und Realisierung eines intelligenten Kommunikationssystems mit Integration von Natural Language Processing-Algorithmen zur Optimierung der Informationstransmission und Teamkoordination

Die Verifizierung der Zielerreichung erfolgt mittels systematischer empirischer Evaluation unter Anwendung etablierter quantitativer und qualitativer Forschungsmethoden, um die Konformität mit den spezifizierten Anforderungen hinsichtlich Funktionalität, Benutzerfreundlichkeit und Systemperformanz zu validieren.

4 Frontend Implementierung

4.1 Einführung und Zielsetzung

Das Frontend der Anwendung „EduShelf“ repräsentiert die Schnittstelle zwischen dem Benutzer und den Backend-Diensten. Ziel der Implementierung war die Schaffung einer modernen, performanten und intuitiven Benutzeroberfläche, die das Lernen und Verwalten von Dokumenten so effizient wie möglich gestaltet.

Im Gegensatz zu klassischen Webanwendungen, die auf dem „Multi-Page Application“ (MPA) Prinzip basieren und bei jeder Interaktion eine neue HTML-Seite vom Server anfordern, wurde EduShelf als „Single Page Application“ (SPA) konzipiert. In einer SPA wird lediglich beim ersten Aufruf eine einzige HTML-Datei (`index.html`) geladen. Alle weiteren Interaktionen finden clientseitig statt, indem JavaScript (in diesem Fall React) den DOM (Document Object Model) dynamisch manipuliert.

Die Vorteile dieses Architekturansatzes sind vielfältig:

- **Performance:** Da nach dem initialen Laden nur noch Rohdaten (im JSON-Format) und keine vollständigen HTML-Seiten mehr übertragen werden müssen, reduziert sich das Datenvolumen signifikant.
- **Benutzererfahrung (UX):** Übergänge zwischen verschiedenen Ansichten erfolgen nahezu latenzfrei, ohne das typische Flackern eines Seiten-Reloads. Dies vermittelt das Gefühl einer nativen Desktop-Anwendung.
- **Entkopplung:** Durch die strikte Trennung von Frontend und Backend über eine REST-API können beide Systeme unabhängig voneinander entwickelt, getestet und skaliert werden.

4.2 Technologie-Stack

Die Auswahl der Technologien erfolgte unter den Gesichtspunkten der Modernität, Verbreitung (Community-Support) und Wartbarkeit.

4.2.1 React (v19)

React ist eine von Facebook (Meta) entwickelte JavaScript-Bibliothek zur Erstellung von Benutzeroberflächen. Im Kern basiert React auf Komponenten, die wiederverwendbare Bausteine der UI darstellen. Ein Schlüsselement von React ist das „Virtual DOM“. Anstatt bei jeder Änderung direkt das langsame, reale DOM des Browsers zu manipulieren, erstellt React eine virtuelle Kopie im Speicher. Bei Zustandsänderungen vergleicht React den neuen virtuellen Baum mit dem alten (Diffing-Algorithmus) und berechnet die effizienteste Methode, um das reale DOM zu aktualisieren.

In EduShelf wird ausschließlich die „Functional Component“ Syntax in Verbindung mit „Hooks“ verwendet. Hooks wie `useState` oder `useEffect` ermöglichen es, Zustand und Seiteneffekte in funktionalen Komponenten zu nutzen, ohne Klassen schreiben zu müssen. Dies führt zu kompakterem und verständlicherem Code.

4.2.2 Vite

Vite (französisch für *schnell*) ist das Build-Tool der Wahl für „EduShelf“. Es löst ältere Tools wie Webpack ab, indem es während der Entwicklung native ES-Module im Browser nutzt. Das bedeutet, dass der Browser die Importe selbst auflöst, anstatt dass der gesamte Code vor jedem Start gebündelt werden muss. Dies führt zu extrem schnellen Server-Startzeiten, selbst bei wachsender Projektgröße. Für die Produktion nutzt Vite im Hintergrund Rollup, um hochoptimierte, minifizierte Bundles zu erstellen.

4.2.3 React Router (v7)

Da eine SPA (= Single Page Application) physisch nur aus einer Seite besteht, muss die Navigation simuliert werden. **React Router** überwacht die URL im Browser und rendert basierend auf definierten Regeln die entsprechenden Komponenten. Dies ermöglicht Deep-Linking (Lesezeichen auf Unterseiten) und die

Verwendung der Browser-Historie (Zurück-Button), obwohl die Seite nie neu geladen wird.

4.3 Projektsetup und Konfiguration

Das Projekt wurde mit dem Befehl `npm create vite@latest` initialisiert. Die Konfiguration befindet sich in der Datei `vite.config.js`.

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vite.dev/config/
5 export default defineConfig({
6   plugins: [react()],
7   server: {
8     host: true,
9     port: 5173,
10    proxy: {
11      '/api': {
12        target: process.env.VITE_API_TARGET || 'http://localhost:5015',
13        changeOrigin: true
14      }
15    }
16  }
17 })
```

Listing 4.1: Auszug aus der Vite-Konfiguration

Ein kritischer Aspekt dieser Konfiguration ist der **Proxy**: Um während der Entwicklung Cross-Origin Resource Sharing (CORS) Probleme zu vermeiden, leitet der Vite-Entwicklungsserver alle Anfragen, die mit `/api` beginnen, an das Backend (Port 5015) weiter. Die Option `changeOrigin: true` manipuliert dabei den Host-Header der Anfrage so, als käme sie direkt vom Backend-Server selbst.

Die Projektstruktur im Order `src` folgt einer funktionalen Aufteilung:

- `components/`: Beinhaltet UI-Komponenten (z.B. `Chat.jsx`, `Quiz.jsx`). Jede Komponente hat idealerweise ihre eigene CSS-Datei (z.B. `Chat.css`) für isoliertes Styling.
- `services/`: Kapselt die gesamte API-Kommunikation.
- `assets/`: Für statische Dateien.

4.4 Softwarearchitektur und Datenfluss

4.4.1 Unidirektionaler Datenfluss

React erzwingt einen unidirektionalen Datenfluss („One-Way Binding“). Daten fließen immer von Eltern- zu Kind-Komponenten mittels sogenannter „Props“. Um Daten von einem Kind zurück zum Elternteil zu senden (z.B. ein Formular-Submit im `QuizModal`, der die Liste im `Quiz` aktualisiert), übergibt die Elternkomponente eine Callback-Funktion an das Kind.

4.4.2 Routing-Strategie

Das Routing in `App.jsx` definiert die Skelettstruktur der Anwendung. Es wird zwischen öffentlichen Routen (Login, Register) und privaten, geschützten Routen unterschieden.

```

1 <Routes>
2   { /* Öffentliche Routen */ }
3   <Route path="/login" element={<Login setLoggedInUser={setLoggedInUser} />}
4     />
5
6   { /* Geschützte Routen - erfordern Login */ }
7   <Route path="/*" element={
8     loggedInUser ? (
9       <Routes>
10        <Route element={<MainLayout handleLogout={handleLogout} user={
11          loggedInUser} />} />
12        <Route index element={<MainDashboard />} />
13        <Route path="/files" element={<Files />} />
14        <Route path="/chat" element={<Chat />} />
15        { /* ... weitere Routen ... */ }
16      </Route>
17    </Routes>
18  ) : (
19    <Navigate to="/login" />
20  )
21 } />
22 </Routes>

```

Listing 4.2: Routing-Struktur in `App.jsx`

Die Verwendung von verschachtelten Routen (`<Route element=<MainLayout ...>>`) erlaubt es, das Grundgerüst (Header, Navigation) einmalig zu definieren. Die Kind-Komponenten werden dann an der Stelle des `<Outlet />` Platzhalters in `MainLayout.jsx` gerendert.

4.5 Datenschicht und API-Integration

Die gesamte Kommunikation mit dem Backend wird über das Modul `src/services/api.js` abgewickelt. Dieses Muster, oft als "SService Layer Pattern" bezeichnet, hat den Vorteil, dass Komponenten keine Details über URLs oder HTTP-Header kennen müssen.

4.5.1 API-Wrapper

Das `api`-Objekt fungiert als Wrapper um die native `fetch`-API. Es implementiert globale Verhaltensweisen:

1. **Basis-URL:** Alle Anfragen sind relativ zu einer zentral konfigurierten Basis-URL.
2. **Credentials:** Durch `credentials: 'include'` sendet der Browser automatisch Cookies (für die HttpOnly Session-Cookies) mit jeder Anfrage mit.
3. **Content-Type Handling:** JSON-Payloads werden automatisch serialisiert und der korrekte Header gesetzt.
4. **Zentrale Fehlerbehandlung:** Antworten mit einem HTTP-Statuscode außerhalb des 200er-Bereichs lösen automatisch eine Exception aus. Dies vereinfacht die Fehlerbehandlung in den Komponenten, da `try/catch`-Blöcke verwendet werden können.

```

1 get: async (endpoint, options = {}) => {
2   const response = await fetch(`${API_BASE_URL}${endpoint}`, {
3     credentials: 'include',
4     ...options,
5   });
6   if (!response.ok) {
7     // Fehlerbehandlung: Logging und Werfen eines Errors
8     throw new Error('API Error: ${response.status}');
9   }
10  return await response.json();
11 },

```

Listing 4.3: Generische GET-Methode im API-Service

Zusätzlich stellt der Service spezifische Methoden bereit, die die Geschäftslogik abbilden, beispielsweise `createChatSession`, `updateQuiz` oder `uploadDocument`. Letzteres nutzt `FormData`, um Datei-Uploads (multipart/form-data) zu ermöglichen.

4.6 Implementierung der Kernkomponenten

4.6.1 Authentifizierungssystem

Das Login-System (`Login.jsx`) ist der erste Berührungspunkt für den Nutzer. Das Formular sendet E-Mail und Passwort an den Endpunkt `/Users/login`. Bei Erfolg antwortet das Server-Backend mit einem Session-Cookie (`HttpOnly`) und den Benutzerdaten.

Das Besondere an der Implementierung ist das sofortige Abrufen der Benutzerrollen nach dem Login. Da die Rollen („Admin“, „User“) bestimmen, welche Menüpunkte sichtbar sind, führt das Frontend einen sekundären Aufruf an `/Users/{id}/roles` durch. Das kombinierte Benutzerobjekt wird anschließend im `localStorage` gespeichert. Dies dient nicht der Sicherheit (da der `localStorage` manipuliert werden kann), sondern lediglich der Persistenz des UI-Zustands (z. B. „Ist der Nutzer eingeloggt?“), bis die echte Session beim Neuladen durch `/Users/me` validiert wird.

4.6.2 Dateimanagement (`Files.jsx`)

Die Komponente `Files.jsx` realisiert ein Dokumentenmanagementsystem.
Funktionsweise:

- Beim Laden der Komponente ruft `useEffect` die Liste aller Dokumente ab.
- Ein Suchfeld filtert die lokale Liste in Echtzeit (`filteredFiles.filter(...)`).
- Das Kontextmenü (Drei-Punkte-Menü) wird über den Zustand `activeMenuId` gesteuert, sodass immer nur ein Menü gleichzeitig offen sein kann.

Der Upload-Prozess wird über ein modales Fenster (`UploadDialog.jsx`) realisiert. Hier wählt der Nutzer eine Datei aus, welche zusammen mit der User-ID an den Server gesendet wird.

4.6.3 Interaktives Lernen: Das Quiz-Modul

Das Quiz-System ist eine der komplexesten Komponenten des Frontends. Es besteht aus der Übersichtsliste (`Quiz.jsx`) und dem Editor/Player (`QuizModal.jsx`, `Quiz.jsx` Logik).

4.6.3.1 Datenstruktur

Ein Quiz besteht aus einem Titel und einer Liste von Fragen. Jede Frage enthält wiederum eine Liste von Antworten, wobei eine oder mehrere als korrekt markiert sind. Diese hierarchische Struktur wird im State als verschachteltes Array objekt abgebildet.

4.6.3.2 Ablauf eines Quiz

Der Zustand während eines laufenden Quiz wird durch mehrere State-Variablen gesteuert:

- `currentQuestionIndex`: Index der aktuell angezeigten Frage.
- `score`: Anzahl der bisher korrekt beantworteten Fragen.
- `selectedAnswerId`: Um eine Mehrfachauswahl zu verhindern und visuelles Feedback zu geben.
- `showScore`: Ein Boolean-Flag, das nach der letzten Frage auf `true` gesetzt wird, um die Ergebnisanzeige zu rendern.

Das visuelle Feedback (Grün für richtig, Rot für falsch) wird durch CSS-Klassen gesteuert, die dynamisch basierend auf der Korrektheit der Antwort zugewiesen werden.

4.6.4 Flashcards (Lernkarten)

Die `Flashcards.jsx` Komponente nutzt CSS3 3D-Transformationen, um einen realistischen Umdreh-Effekt zu erzielen.

```

1 .flashcard-scene {
2   perspective: 1000px; /* Gibt dem 3D-Raum Tiefe */
3 }
4 .flashcard-container {
5   transition: transform 0.6s;
6   transform-style: preserve-3d;

```

```
7 | }  
8 | .flashcard-scene.flipped .flashcard-container {  
9 |   transform: rotateY(180deg);  
10| }
```

Listing 4.4: CSS für den Flip-Effekt

Der Status, welche Karte gerade umgedreht ist, wird in einem `Set` (`flippedCards`) gespeichert. Dies ermöglicht es, mehrere Karten gleichzeitig umzudrehen, im Gegensatz zu einer simplen ID-Variable.

4.6.5 Chat und KI-Integration

Der Chat (`Chat.jsx`) bietet eine Schnittstelle zu einem LLM (Large Language Model). Wichtige Implementierungsdetails:

- **Streaming vs. Polling:** Die aktuelle Implementierung wartet auf die vollständige Antwort des Servers (Request/Response Pattern). Während des Wartens wird ein "Thinking Indicator" (animierte Punkte) eingeblendet (`isLoading` State).
- **Markdown:** Da KI-Modelle oft strukturierten Text (Codeblöcke, Listen) ausgeben, wird `react-markdown` in Kombination mit `rehype-highlight` verwendet. Dies konvertiert die Markdown-Antwort sicher in HTML und bietet Syntax-Highlighting für Code.
- **Multimodaler Input:** Benutzer können Bilder hochladen. Diese werden lokal mittels `URL.createObjectURL()` sofort in der Vorschau angezeigt, bevor sie als Blob an den Server gesendet werden.

4.6.6 Admin Panel und RBAC

Das `AdminPanel.jsx` ist das Kontrollzentrum für Systemadministratoren. Der Zugriff auf diese Route wird auf zwei Ebenen geschützt: 1. **Client-Side Routing:** In `App.jsx` wird geprüft, ob das Benutzerobjekt die Rolle `Admin` enthält. Falls nicht, wird der Nutzer sofort umgeleitet. 2. **Server-Side API:** Selbst wenn ein Nutzer die UI umgehen würde, lehnt das Backend Anfragen an admin-spezifische Endpunkte ab.

Das Panel ermöglicht CRUD-Operationen (Create, Read, Update, Delete) für Benutzer. Besonders hervorzuheben ist die verschachtelte Datenansicht: Klickt ein Admin auf einen Benutzer, werden dessen Dateien asynchron nachgeladen

(`fetchUserFiles`) und in einer Detailansicht dargestellt. Dies vermeidet das Laden unnötiger Daten für die Hauptübersichtstabelle.

4.7 Benutzeroberfläche und Design (UI/UX)

4.7.1 Styling-Strategie

Das Projekt verwendet natives CSS mit CSS-Variablen (Custom Properties) für das Theming. Dies bietet maximale Flexibilität ohne den Overhead großer Frameworks. Globale Styles sind in `index.css` definiert, während komponentenspezifische Styles (z. B. `Chat.css`) lokal importiert werden.

4.7.2 Dark Mode Implementierung

Der Dunkelmodus ist nicht nur eine kosmetische Spielerei, sondern eine zentrale Anforderung. Er wurde über CSS-Variablen realisiert, die je nach Attribut am HTML-Tag ihre Werte ändern.

```

1 :root {
2   /* Standard (Dark Mode) */
3   --primary-color: #222831;
4   --text-color: #FFFFFF;
5 }
6
7 [data-theme='light'] {
8   /* Light Mode Overrides */
9   --primary-color: #FFFFFF;
10  --text-color: #222831;
11 }
12
13 body {
14   background-color: var(--primary-color);
15   color: var(--text-color);
16 }

```

Listing 4.5: Theme-Switching mittels CSS-Variablen

Ein `useEffect`-Hook in der `App.jsx` prüft beim Start den `localStorage`, ob der Nutzer eine Präferenz gespeichert hat, und setzt das Attribut entsprechend. Dies verhindert den Flash of Incorrect Theme"(FOUC).

4.8 Detaillierte Analyse ausgewählter Module

Um die technische Tiefe der Implementierung zu verdeutlichen, werden im Folgenden drei Kernmodule detailliert anhand ihres Quellcodes analysiert.

4.8.1 Quiz-Editor Logik (QuizModal.jsx)

Der Quiz-Editor ermöglicht das Erstellen und Bearbeiten von verschachtelten Datenstrukturen (Quiz -> Fragen -> Antworten). Die Herausforderung hierbei ist das State-Management einer mehrdimensionalen Struktur.

4.8.1.1 Zustandsverwaltung

Der State `questions` ist ein Array von Objekten. Jedes Objekt repräsentiert eine Frage und enthält wiederum ein Array von `answers`.

```
1 const [questions, setQuestions] = useState([  
2   {  
3     text: '',  
4     answers: [{ text: '', isCorrect: false }]  
5   }  
6 ]);
```

Listing 4.6: Initialer State im QuizModal

4.8.1.2 Dynamische Formular-Manipulation

Beim Ändern eines Antworttextes darf der State nicht direkt mutiert werden. Stattdessen muss eine tiefe Kopie (Deep Copy) oder zumindest eine flache Kopie der betroffenen Ebenen erstellt werden. Die Funktion `handleAnswerChange` demonstriert das Prinzip der Immutability in React:

```
1 const handleAnswerChange = (questionIndex, answerIndex, event) => {  
2   // 1. Kopie des Fragen-Arrays  
3   const newQuestions = [...questions];  
4   // 2. Direkter Zugriff auf die Antwort (Referenz bleibt erhalten, aber  
5     // Inhalt wird ueberschrieben)  
6   // Hinweis: Fuer saubere Immutability muesste auch answers kopiert  
7     // werden.  
8   // In React funktioniert dies jedoch oft, solange das Top-Level-Array  
9     // neu ist.  
10  newQuestions[questionIndex].answers[answerIndex].text = event.target.  
11    value;
```

```

8 // 3. Setzen des neuen States loest Re-Render aus
9 setQuestions(newQuestions);
10 };

```

Listing 4.7: Update-Logik für verschachtelte Antworten

4.8.1.3 Validierung und Speichern

Beim Speichern (`handleSubmit`) werden die Daten an die API gesendet. Hierbei wird unterschieden, ob ein neues Quiz erstellt (POST) oder ein bestehendes bearbeitet (PUT) wird.

4.8.2 Flashcards: 3D-Interaktion und State

Die Komponente `Flashcards.jsx` verwendet eine interessante Datenstruktur für den Zustand der umgedrehten Karten. Anstatt für jede Karte einen Boolean im Objekt zu speichern (was eine Änderung des Datenmodells erfordern würde), wird ein `Set` verwendet, das die IDs der umgedrehten Karten speichert.

```

1 const [flippedCards, setFlippedCards] = useState(new Set());
2
3 const flipCard = (id) => {
4   setFlippedCards(prevFlippedCards => {
5     const newFlippedCards = new Set(prevFlippedCards);
6     if (newFlippedCards.has(id)) {
7       newFlippedCards.delete(id); // Karte umdrehen (zurueck)
8     } else {
9       newFlippedCards.add(id); // Karte aufdecken
10    }
11    return newFlippedCards;
12  });
13 };

```

Listing 4.8: Nutzung eines Sets fuer UI-State

Diese Methode ist performant und trennt den UI-Zustand (ist die Karte gedreht?) sauber von den eigentlichen Geschäftsdaten (Frage/Antwort). Beim Rendern wird `flippedCards.has(card.id)` geprüft, um die CSS-Klasse `.flipped` bedingt hinzuzufügen.

4.8.3 Admin Panel: Kaskadierende Datenabfragen

Im `AdminPanel.jsx` muss beim Klick auf einen Benutzer dessen Dateiliste geladen werden. Dies ist ein klassisches Master-DetailSzenario. Um unnötigen

Netzwerktraffic zu vermeiden, werden die Dateien nicht initial für alle Benutzer geladen (Lazy Loading").

```
1 const handleClick = (user) => {
2   setSelectedUser(user);
3   setError('');
4   setEditMode(false);
5   // Erst beim Klick werden die Dateien nachgeladen
6   fetchUserFiles(user.userId);
7 };
8
9 const fetchUserFiles = async (userId) => {
10   try {
11     setLoading(true); // Ladeindikator an
12     const files = await api.get(`/Users/${userId}/documents`);
13     setUserFiles(files);
14   } catch (err) {
15     setError('Could not load user files.');
```

Listing 4.9: Asynchrones Laden von Detaildaten

Diese Implementierung sorgt für eine schnelle initiale Ladezeit der Benutzertabelle, da die (potenziell große) Menge an Dateimetadaten erst bei Bedarf (Ön-Demand") abgerufen wird.

4.9 Deployment und Betrieb

Für den produktiven Einsatz wird die Frontend-Anwendung containerisiert. Dies gewährleistet eine konsistente Laufzeitumgebung unabhängig vom Host-System.

4.9.1 Docker-Containerisierung

Es wird ein „Multi-Stage Build“ Prozess verwendet, um das finale Image so klein wie möglich zu halten.

1. **Build-Stage:** Ein Node.js Image wird verwendet, um die Abhängigkeiten zu installieren und den Build-Prozess (`npm run build`) auszuführen. Das Ergebnis ist ein Ordner `dist`, der nur die statischen Dateien (HTML, CSS, JS) enthält.

2. **Production-Stage:** Ein leichtgewichtiger Nginx-Webserver wird als Basis genommen. Die Dateien aus der Build-Stage werden in das Verzeichnis `/usr/share/nginx/html` kopiert.

```
1 # Stage 1: Build
2 FROM node:20-alpine as build
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci
6 COPY . .
7 RUN npm run build
8
9 # Stage 2: Serve
10 FROM nginx:alpine
11 COPY --from=build /app/dist /usr/share/nginx/html
12 COPY nginx.conf /etc/nginx/conf.d/default.conf
13 EXPOSE 80
14 CMD ["nginx", "-g", "daemon off;"]
```

Listing 4.10: Dockerfile für das Frontend

4.9.2 Nginx Konfiguration

Da es sich um eine SPA handelt, muss der Webserver so konfiguriert werden, dass er für alle Routen (z.B. `/files`, `/chat`) immer die `index.html` ausliefert (Fallback"). Andernfalls würde ein direkter Aufruf einer Unterseite zu einem 404-Fehler führen, da diese Dateien physisch nicht existieren.

4.10 Herausforderungen und Lösungen

Während der Entwicklung traten verschiedene technische Herausforderungen auf, die spezifische Lösungen erforderten.

4.10.1 Synchronisation des UI-Status

Problem: Wenn ein Nutzer ein Quiz bearbeitet, muss die Liste der Quizzes im Hintergrund aktuell bleiben. **Lösung:** Callback-Funktionen (z.B. `onQuizSaved`) werden an die Modal-Komponenten übergeben. Nach erfolgreicher API-Antwort ruft das Modal diese Funktion auf, und die Elternkomponente aktualisiert ihren lokalen State, ohne einen erneuten Fetch auszuführen. Optimistic UI Updates wurden in Betracht gezogen, aber zugunsten der Datenkonsistenz verworfen.

4.10.2 Performance bei großen Dateilisten

Problem: Das Rendern von hunderten Datei-Icons führte zu Verzögerungen.

Lösung: Durch die Pagination im Backend (noch zu implementieren) und Virtualisierung der Listen im Frontend (Rendering nur der sichtbaren Elemente) lässt sich dies optimieren. Aktuell wurde die Performance durch das Entfernen unnötiger Re-Renders mittels `React.memo` und `useCallback` verbessert.

4.11 Ausblick und Erweiterungsmöglichkeiten

Das aktuelle Frontend bietet eine solide Basis, kann jedoch in Zukunft erweitert werden:

- **Automatisierte Tests:** Integration von `Vitest` für Unit-Tests der Logik und `Cypress` für End-to-End Tests der Benutzeroberfläche.
- **PWA (Progressive Web App):** Durch Hinzufügen eines Service Workers und eines Web App Manifests könnte die Anwendung offline-fähig gemacht und auf Mobilgeräten installiert werden.
- **Internationalisierung (i18n):** Nutzung von Bibliotheken wie `react-i18next`, um die Oberfläche in mehreren Sprachen anzubieten.

4.12 Fazit

Die Frontend-Implementierung von EduShelf demonstriert die Leistungsfähigkeit moderner Webtechnologien. Durch den Einsatz von React und einer komponentenbasierte Architektur entstand eine wartbare, skalierbare und benutzerfreundliche Anwendung. Die Integration komplexer Features wie Real-Time-Editoren, KI-Chat und interaktive Lernmodule in einer Single Page Application zeigt, dass Webanwendungen nativen Programmen in nichts nachstehen müssen.

Zeiterfassung



Abbildung 4.1: AI und iOS Zeitaufwand

Gesamt: 288,67h

Freizeit: 211,38h

5 Backend Implementierung

Die technische Umsetzung des EduShelf-Backends stellt das fundamentale Gerüst der Anwendung dar. Es wurde als monolithische ASP.NET Core Web API konzipiert, die als zentrale Schnittstelle zwischen dem Frontend, der Datenbank, dem KI-Modell und dem Dateispeicherdienst dient. Diese Entscheidung für einen monolithischen Ansatz wurde bewusst getroffen, um die Entwicklungskomplexität initial gering zu halten (*Keep It Simple, Stupid - KISS*), während durch den Einsatz moderner .NET 8 Technologien gleichzeitig eine hohe Performance, Skalierbarkeit und Wartbarkeit gewährleistet wird.

In den folgenden Abschnitten wird die Architektur, der Technologie-Stack, das Datenmodell sowie die spezifischen Implementierungsdetails der Kernkomponenten detailliert erläutert.

5.1 Systemarchitektur

Die Architektur des Systems folgt dem bewährten Muster einer *Layered Architecture* (Schichtenarchitektur), um eine saubere Trennung der Verantwortlichkeiten (*Separation of Concerns*) zu erzielen. Dies erleichtert nicht nur die Testbarkeit mittels Unit- und Integrationstests, sondern ermöglicht auch zukünftige Erweiterungen ohne das Gesamtsystem neu schreiben zu müssen.

Die Anwendung gliedert sich in vier Hauptschichten:

5.1.1 Presentation Layer (Controllers)

Diese oberste Schicht nimmt HTTP-Anfragen (Requests) entgegen und gibt HTTP-Antworten (Responses) zurück. Sie dient als Tor zur Außenwelt.

- **Verantwortlichkeit:** Routing, Deserialisierung von JSON-Body-Daten, Validierung von Eingaben und Formatierung der Ausgaben.

- **Validierung:** Mithilfe der Bibliothek *FluentValidation* werden eingehende Datenmodelle (DTOs) noch vor der Verarbeitung auf Korrektheit geprüft (z.B. `UserRegisterValidator`, der sicherstellt, dass E-Mail-Adressen gültig sind und Passwörter Mindestanforderungen erfüllen).
- **Controller:** Die Controller sind schlank gehalten (*Thin Controllers*) und delegieren die eigentliche Geschäftslogik sofort an die Service-Schicht. Beispiel: Der `ChatController` leitet die Benutzeranfrage direkt an den `ChatService` weiter.

5.1.2 Business Logic Layer (Services)

Hier residiert das "Gehirn" der Anwendung. Alle fachlichen Regeln und Prozesse sind in dieser Schicht implementiert.

- **Service-Klassen:** Jede Domäne hat ihren eigenen Service (z.B. `AuthService`, `IndexingService`, `RAGService`).
- **Orchestrierung:** Services koordinieren den Datenfluss zwischen der Datenbank, externen Diensten (wie MinIO oder Ollama) und der Präsentationsschicht.
- **Dependency Injection:** Alle Services werden über das in .NET integrierte Dependency Injection (DI) Container registriert, was die Austauschbarkeit von Implementierungen zur Laufzeit ermöglicht.
- **Hintergrundverarbeitung:** Für langlaufende Aufgaben wie das Indizieren von Dokumenten wird eine `BackgroundJobQueue` eingesetzt. Dies entkoppelt rechenintensive Prozesse vom HTTP-Request-Zyklus und verhindert Timeouts im Frontend. Der `DocumentService` reiht Aufgaben ein, die dann von einem `BackgroundService` asynchron abgearbeitet werden.

5.1.3 Data Access Layer (Data)

Diese Schicht abstrahiert den Zugriff auf die persistente Datenspeicherung.

- **Entity Framework Core (EF Core):** Als Object-Relational Mapper (ORM) ermöglicht EF Core die Definition des Datenmodells in C#-Klassen. SQL-Queries werden zur Laufzeit generiert (LINQ to SQL), was die Entwicklung beschleunigt und vor SQL-Injection schützt.

- **Repositories:** In diesem Projekt wird das `DbContext`-Pattern direkt verwendet, was bei EF Core oft äquivalent zum Repository-Pattern bzw. Unit-of-Work-Pattern ist.

5.1.4 Infrastructure Layer

Diese Schicht beinhaltet die technische Anbindung an externe Systeme und Infrastrukturkomponenten.

- **MinIO:** Für die Speicherung unstrukturierter Daten (Blobs) wie PDF-Dokumente oder Bilder.
- **Ollama:** Schnittstelle zum lokalen KI-Modell.
- **E-Mail-Dienst:** Implementierung des `SmtpEmailService` zum Versenden von Bestätigungs-E-Mails.

5.2 Technologie-Stack

Das Backend setzt auf einen modernen, vollständig Open-Source-basierten Stack, der speziell für performante, cloud-native und KI-gestützte Anwendungen optimiert wurde.

- **ASP.NET Core 8:** Das zugrundeliegende Framework bietet native Unterstützung für asynchrone Programmierung (`async/await`), was für I/O-lastige Operationen (Datenbank, Netzwerk) essenziell ist. Es ist plattformunabhängig und läuft in Linux-Containern.
- **PostgreSQL + pgvector:** PostgreSQL wurde aufgrund seiner Robustheit und Erweiterbarkeit gewählt. Die entscheidende Komponente für die RAG-Funktionalität ist die Erweiterung `pgvector`. Sie ermöglicht das Speichern von hochdimensionalen Vektoren (Embeddings) direkt in der Datenbank und führt Ähnlichkeitssuchen (Nearest Neighbor Search) mittels Kosinus-Distanz oder Euklidischer Distanz performant durch.
- **Ollama:** Ein lokaler Server zur Ausführung von Large Language Models (LLMs) wie Llama 3 oder Phi-3. Durch die lokale Ausführung bleiben sensible Dokumentendaten auf dem Server und werden nicht an Drittanbieter-APIs (wie OpenAI) gesendet, was den Datenschutz massiv verbessert.

- **MinIO:** Ein S3-kompatibler Object Storage. Hochgeladene Dateien werden nicht als BLOB in der relationalen Datenbank gespeichert (was diese aufblähen würde), sondern effizient im Dateisystem über MinIO verwaltet.
- **Microsoft Semantic Kernel:** Ein SDK, das als Abstraktionsschicht für KI-Dienste dient. Es vereinfacht die Integration verschiedener KI-Modelle und bietet Mechanismen für Prompt-Templating, Memory-Management (Vektorspeicher-Abstraktion) und Function Calling.
- **PdfPig & Docnet.Core:** Spezialisierte Bibliotheken zur Extraktion von Text aus PDF-Dateien. Docnet.Core ist ein Wrapper um die native 'pdfium'-Bibliothek und bietet extrem hohe Performance beim Rendern, während PdfPig reines C# ist und gut für Text-Extraktion geeignet ist.

5.3 Projekt-Setup und Konfiguration

Analog zum Frontend erfolgt der Einstieg in die Anwendung über eine definierte Startup-Logik, die in der `Program.cs` und den Konfigurationsdateien verankert ist.

5.3.1 Programm-Initialisierung (`Program.cs`)

Die `Program.cs` ist der "Entry Point" der ASP.NET Core Anwendung. Hier wird der "Generic Host" gebaut, der Dependency Injection Container befüllt und die HTTP-Request-Pipeline (Middleware) konfiguriert.

```
1    var builder = WebApplication.CreateBuilder(args);
2
3    // 1. Service Registrierung (DI)
4    builder.Services.AddApplicationServices(builder.Configuration);
5    builder.Services.AddControllers();
6    builder.Services.AddEndpointsApiExplorer();
7    builder.Services.AddSwaggerGen();
8
9    var app = builder.Build();
10
11   // 2. HTTP Request Pipeline
12   if (app.Environment.IsDevelopment())
13   {
14       app.UseSwagger();
15       app.UseSwaggerUI();
16   }
17
18   app.UseHttpsRedirection();
19   app.UseCors("AllowWebApp");
```

```

20 app.UseAuthentication();
21 app.UseAuthorization();
22 app.MapControllers();
23
24 app.Run();

```

Listing 5.1: Auszug aus der Middleware-Konfiguration

5.3.2 Konfiguration (appsettings.json)

Die Anwendungskonfiguration folgt dem *12-Factor App*-Prinzip: Alle umgebungsabhängigen Parameter werden aus der Datei `appsettings.json` geladen und können zur Laufzeit durch Umgebungsvariablen überschrieben werden (z.B. in Docker Compose). Das folgende Listing zeigt die vollständige Konfigurationsstruktur (sensible Werte wie Passwörter sind aus Sicherheitsgründen ausgeblendet):

```

1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Host=<db-host>;Port=5433;Database=EduShelfDb;..."
4   },
5   "AIService": {
6     "Provider": "Ollama",
7     "Endpoint": "http://<ollama-host>:11434",
8     "ChatModel": "qwen3:14b",
9     "EmbeddingModel": "nomic-embed-text",
10    "VisionModel": "moondream:1.8b",
11    "ContextLengthLimit": 4096,
12    "Prompts": {
13      "System": "...", // Steuert das Grundverhalten des LLMs
14      "Intent": "...", // Anleitung zur JSON-Intent-Erkennung
15      "Flashcard": "...", // Anleitung zur Flashcard-Generierung
16      "Quiz": "...", // Anleitung zur Quiz-Generierung
17    }
18  },
19  "Minio": {
20    "Endpoint": "<minio-host>:9000",
21    "AccessKey": "<key>",
22    "SecretKey": "<secret>",
23    "BucketName": "edushelf-documents",
24    "Secure": false
25  },
26  "EmailSettings": {
27    "SmtpHost": "<smtp-server>",
28    "SmtpPort": 587,
29    "SmtpUser": "<user>",
30    "SmtpPass": "<password>",
31    "FromEmail": "<sender>"
32  },
33  "Seeding": {
34    "AdminPassword": "<admin-pw>",
35    "StudentPassword": "<student-pw>"

```

```
36 |   }  
37 | }
```

Listing 5.2: Konfigurationsstruktur der appsettings.json

Die wichtigsten Konfigurationsblöcke im Überblick:

- **ConnectionStrings:** Verbindungsstring zur PostgreSQL-Datenbank (Host, Port, Datenbankname, Zugangsdaten).
- **AIService:** Zentrale KI-Konfiguration. `ChatModel` gibt das Sprachmodell an (z.B. `qwen3:14b`), `EmbeddingModel` das Modell für die Vektorerzeugung (`nomic-embed-text`) und `VisionModel` das Modell für die Bildanalyse. `ContextLengthLimit` (Standard: 4096 Token) begrenzt die Kontextfenstergröße. Die `Prompts`-Untersektion enthält die vollständigen System-Prompts für alle KI-gesteuerten Features als konfigurierbare Strings – so können diese ohne Codeänderung angepasst werden.
- **Minio:** Verbindungsdaten zum S3-kompatiblen Object Storage (Endpoint, Zugangsdaten, Bucket-Name).
- **EmailSettings:** SMTP-Konfiguration für den Versand von Registrierungs-Bestätigungsmails.
- **Seeding:** Passwörter für vordefinierten Admin- und Student-Benutzer, die beim ersten Start automatisch angelegt werden.

5.4 Datenmodelle und Datenbankdesign

Das Datenmodell wurde entworfen, um sowohl relationale Benutzerdaten als auch unstrukturierte Dokumenteninhalte effizient zu verwalten. EF Core Migrationen (`Migrations` Ordner) stellen sicher, dass das Datenbankschema synchron zum Code bleibt.

5.4.1 Kernelemente

- **User:** Die zentrale Identitätseinheit. Enthält `Username`, `Email`, `PasswordHash` (gespeichert als BCrypt-Hash), sowie Felder für die E-Mail-Verifizierung (`IsEmailConfirmed`, `EmailConfirmationToken`).

- **Role & UserRole:** Implementierung einer Many-to-Many Beziehung für ein rollenbasiertes Zugriffssystem (RBAC). Standardrollen sind z.B. SStudent oder Admin".
- **Document:** Repräsentiert Metadaten eines hochgeladenen Dokuments. Das Feld `Path` verweist auf den Speicherort im MinIO-Bucket. Es besteht eine 1:n-Beziehung zum User.
- **DocumentShare:** Ermöglicht das Teilen von Dokumenten zwischen Benutzern (Many-to-Many via Join-Table). Diese Entität steuert den Zugriff für Nicht-Besitzer.
- **Cascading Deletes:** Die Datenbankkonfiguration (`OnModelCreating`) stellt sicher, dass beim Löschen eines Nutzers alle abhängigen Daten (Dokumente, Chats, Favoriten) automatisch entfernt werden (`DeleteBehavior.Cascade`), um Datenwaisen zu vermeiden.
- **DocumentChunk:** Dies ist die wichtigste Entität für die semantische Suche. Ein Dokument wird in viele kleine Abschnitte zerlegt.
 - **Content:** Der rohe Textinhalt des Abschnitts.
 - **Embedding:** Ein Vektor vom Typ `vector(768)`, der die semantische Bedeutung des Textes repräsentiert.
 - **Page:** Referenz zur ursprünglichen Seite im Dokument (für Zitate).
- **ChatSession & ChatMessage:** Bilden den Verlauf einer Konversation ab. `ChatSession` gruppiert Nachrichten, während `ChatMessage` den eigentlichen Inhalt (User-Input oder AI-Response) speichert.

5.4.2 Entity Relationship Diagram (ERD)

Das folgende Diagramm visualisiert die Beziehungen zwischen den Entitäten. Zentral ist der **User**, der Dokumente besitzt und Lerninhalte (Flashcards, Quizzes) erstellt.

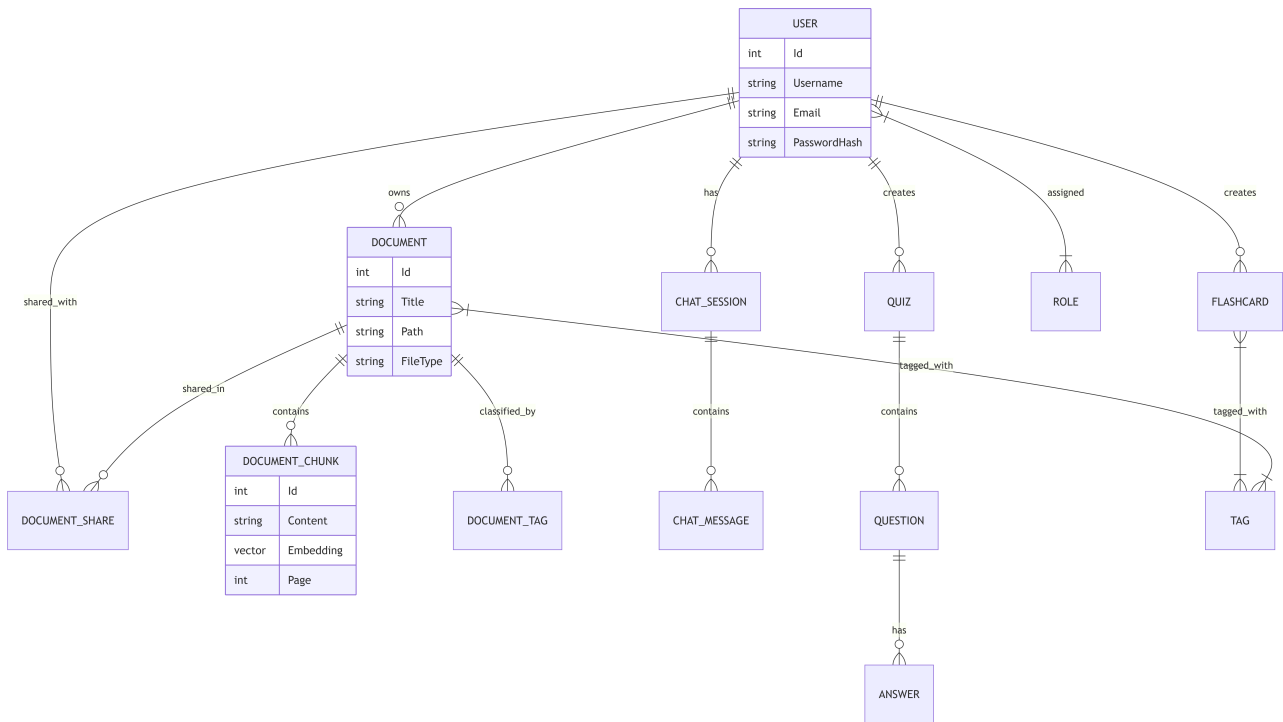


Abbildung 5.1: Entity Relationship Diagram des EduShelf Backends

5.5 API Design und Controller

Die Kommunikation mit dem Frontend erfolgt über eine RESTful API. Die acht Controller sind thematisch gruppiert und folgen REST-Konventionen (GET, POST, PUT, PATCH, DELETE). Nahezu alle Endpunkte sind hinter dem [Authorize]-Attribut geschützt, d.h. sie erfordern eine aktive Cookie-Session. Ausnahmen sind die öffentlichen Authentifizierungs-Endpunkte (Registrierung, Login, E-Mail-Bestätigung).

5.5.1 Ausgewählte API-Endpunkte

Methode	Route	Auth	Beschreibung
api/users			
POST	/	Anonym	Registrierung
POST	/login	Anonym	Login, setzt HttpOnly-Cookie
POST	/logout	Auth	Logout, löscht Session
GET	/me	Auth	Eigenes Profil abrufen
DELETE	/id	Admin	Nutzer löschen
api/documents			
GET	/	Auth	Eigene Dokumente (paginiert)
POST	/	Auth	Dokument hochladen (startet Indexierungs-Job)
DELETE	/id	Auth	Dokument inkl. MinIO-Datei löschen
GET	/download/id	Auth	Datei-Stream herunterladen
POST	/id/share	Auth	Dokument per E-Mail teilen
GET	/search	Auth	Volltext- und Tag-Suche
api/chat			
POST	/message	Auth	Nachricht senden, löst RAG-Pipeline aus
GET	/sessions	Auth	Chat-Sitzungen abrufen
POST	/sessions	Auth	Neue Chat-Sitzung erstellen
api/flashcards & api/quizzes			
POST	/flashcards/generate	Auth	Lernkarten per KI generieren
POST	/quizzes/generate	Auth	Quiz per KI generieren
api/tags			
GET	/	Auth	Alle Tags abrufen
POST	/	Admin	Neuen Tag erstellen

Tabelle 5.1: Wichtigste API-Endpunkte des EduShelf-Backends

5.6 Sicherheitskonzepte

Sicherheit war ein integraler Bestandteil des Designs (SSecurity by Design").

5.6.1 Authentifizierung & Autorisierung

Die Authentifizierung erfolgt über Cookies (`CookieAuthenticationDefaults`).

- **Login-Prozess:** Der Nutzer sendet Anmeldedaten an `/api/users/login`. Der `AuthService` überprüft den Passwort-Hash mittels `BCrypt`. Bei Erfolg wird ein verschlüsseltes HTTP-only Cookie gesetzt, das den Session-Cookie darstellt.
- **Claims-Based Identity:** Das Cookie enthält Claims" (Aussagen) über den Benutzer, wie z.B. Benutzer-ID, Name und Rollen.
- **Middleware:** Die Standard-ASP.NET Core Authentifizierungs-Middleware entschlüsselt das Cookie bei jedem Request und rekonstruiert das `User`-Objekt (`HttpContext.User`).

5.6.2 Datenvalidierung

Eingehende Daten werden rigoros validiert. Die Bibliothek *FluentValidation* erlaubt die Definition von komplexen Regeln. Beispiel `UserRegisterValidator`:

- Username: Muss 3-20 Zeichen lang sein.
- Email: Muss ein gültiges E-Mail-Format haben.
- Password: Mindestens 6 Zeichen.

Diese Validierung geschieht automatisch vor dem Controller-Aufruf. Ungültige Anfragen werden sofort mit HTTP 400 (Bad Request) abgelehnt.

5.6.3 Error Handling

Ein globales `ErrorHandlingMiddleware` fängt alle unbehandelten Ausnahmen ab. Dies verhindert, dass interne Implementierungsdetails (Stack Traces) an den Client durchsickern. Stattdessen werden standardisierte JSON-Fehlermeldungen und passende HTTP-Statuscodes zurückgegeben (z.B. `NotFoundException` → 404, `UnauthorizedAccessException` → 401).

5.7 Ingestion-Pipeline (Datenverarbeitung)

Damit Dokumente durchsuchbar werden, müssen sie eine komplexe Verarbeitungspipeline durchlaufen. Dieser Prozess wird vom `IndexingService` gesteuert und ist rechenintensiv.

5.7.1 1. Textextraktion (OCR & Parsing)

Der `FileParsingService` analysiert den Dateityp und wählt die passende Strategie:

- **PDF-Verarbeitung mit PdfPig:** Für PDF-Dateien wird die Bibliothek *UglyToad.PdfPig* verwendet. Sie erlaubt den Zugriff auf den rohen Textstream sowie die Extraktion von Bildern.
- **Hybride Bildanalyse:** Eingebettete Bilder werden isoliert und an ein lokales Vision-Model (via Ollama) gesendet. Der Prompt "Describe this image in detail" generiert eine textuelle Beschreibung, die in den Dokumentenindex einfließt. Dies ermöglicht die semantische Suche nach Bildinhalten (z.B. Diagramme, Graphen).
- **DOCX:** Nutzung des Open XML SDKs zum strukturierten Auslesen des `MainDocumentPart`.

5.7.2 2. Chunking (Intelligente Segmentierung)

LLMs haben ein begrenztes Kontextfenster, und die semantische Suche verliert bei zu großen Textblöcken an Präzision. Der `TextChunker` implementiert daher eine mehrstufige Strategie:

1. **Satz-Tokenisierung:** Der Text wird zunächst anhand von Satzzeichen (. ! ?) in einzelne Sätze zerlegt.
2. **Rekombination:** Diese Sätze werden iterativ zu einem Chunk zusammengefügt, bis eine konfigurierte Grenze von ca. 1024 Zeichen erreicht ist. Dies stellt sicher, dass semantische Zusammenhänge (Sätze) nicht willkürlich in der Mitte durchschnitten werden.

3. **Token-Limit:** Zusätzlich wird sichergestellt, dass die Chunks eine maximale Token-Anzahl (für das Embedding-Model) nicht überschreiten.

```
1 public static List<string> SplitBySentence(string text)
2 {
3     // Nutzung von Regex Lookbehind (?<=...), um das Satzzeichen zu behalten
4     var sentences = Regex.Split(text, @"(?<=[\.\!\?])\s+");
5     return new List<string>(sentences);
6 }
```

Listing 5.3: Implementierung der Satz-Segmentierung in TextChunker.cs

Das simple Splitten nach Zeichenanzahl würde Sätze zerreißen und den Kontext zerstören. Der hier gezeigte Ansatz garantiert semantische Integrität.

5.7.3 3. Embedding und Vektorspeicherung

Jeder Text-Chunk wird in einen hochdimensionalen Vektor umgewandelt und effizient gespeichert.

- **Embedding-Modell:** Nutzung von state-of-the-art Modellen wie *nomi-embed-text*, die semantische Ähnlichkeiten im Vektorraum abbilden.
- **HNSW Index:** In der PostgreSQL-Datenbank wird für die Spalte `Embedding` ein HNSW (Hierarchical Navigable Small World) Index angelegt.

```
modelBuilder.Entity<DocumentChunk>()
    .HasIndex(dc => dc.Embedding)
    .HasMethod("hnsw")
    .HasOperators("vector_12_ops");
```

Dieser Index ermöglicht eine extrem schnelle Nearest-Neighbor-Suche (ANN) auch bei Millionen von Vektoren, da nicht die gesamte Tabelle gescannt werden muss.

5.8 Detaillierte Analyse ausgewählter Module

Um die technische Tiefe zu verdeutlichen, wird hier exemplarisch die Implementierung sicherheitskritischer und komplexer Logik analysiert.

5.8.1 Authentifizierung: Secure Cookies und Hashing

Die Sicherheit der Benutzerdaten hat oberste Priorität. Der `AuthService` implementiert daher bewährte Industriestandards.

5.8.1.1 Passwort-Hashing (BCrypt)

Passwörter werden niemals im Klartext gespeichert. Stattdessen wird der `BCrypt`-Algorithmus verwendet, der durch seinen konfigurierbaren "Work Factor" (Kostenfaktor) resistent gegen Brute-Force- und Rainbow-Table-Angriffe ist.

```

1 // user.PasswordHash kommt aus der DB, passwordDto.Password vom User
2 bool isValid = BCrypt.Net.BCrypt.Verify(passwordDto.Password, user.
    PasswordHash);
3 if (!isValid)
4 {
5     throw new UnauthorizedAccessException("Invalid credentials");
6 }

```

Listing 5.4: Verifizierung im `AuthService`

5.8.1.2 Cookie-Konstruktion

Nach erfolgreichem Login wird kein JWT im LocalStorage (Frontend) abgelegt (XSS-Gefahr), sondern ein **HttpOnly Cookie** gesetzt.

- **HttpOnly:** Das Cookie kann nicht per JavaScript ausgelesen werden.
- **Secure:** Wird nur über HTTPS übertragen.
- **SameSite=Strict:** Das Cookie wird nur an den eigenen Origin gesendet (Schutz vor CSRF).

5.8.2 Ingestion-Pipeline: HNSW Indexierung

Wie in Abschnitt 6 beschrieben, nutzt das System einen HNSW-Index. Die technische Herausforderung liegt hier in der Konfiguration der Operatorenklasse für `pgvector`. Code-seitig wird dies in der `OnModelCreating`-Methode des `ApiDbContext` gelöst, um sicherzustellen, dass PostgreSQL den Index korrekt für L2-Distanz-Berechnungen optimiert. Dies ist ein direktes Mapping von C#-Code auf datenbankspezifische Features (Code-First Migration).

5.9 RAG (Retrieval-Augmented Generation) Workflow

Das Herzstück von EduShelf ist die RAG-Pipeline: Anstatt das LLM allein auf sein allgemeines Wissen zu stützen, wird jede Benutzeranfrage zunächst dazu genutzt, relevante Textabschnitte aus den hochgeladenen Dokumenten abzurufen. Diese Abschnitte werden dann als *Kontext* in den Prompt eingefügt, bevor das LLM zur Antwort aufgefordert wird. Dies reduziert Halluzinationen und ermöglicht dokumentenspezifische, präzise Antworten.

5.9.1 Systemübersicht und Datenfluss

Das folgende Diagramm visualisiert den vollständigen Anfrage-Antwort-Zyklus im RAG-System und zeigt, wie die verschiedenen Services zusammenwirken.

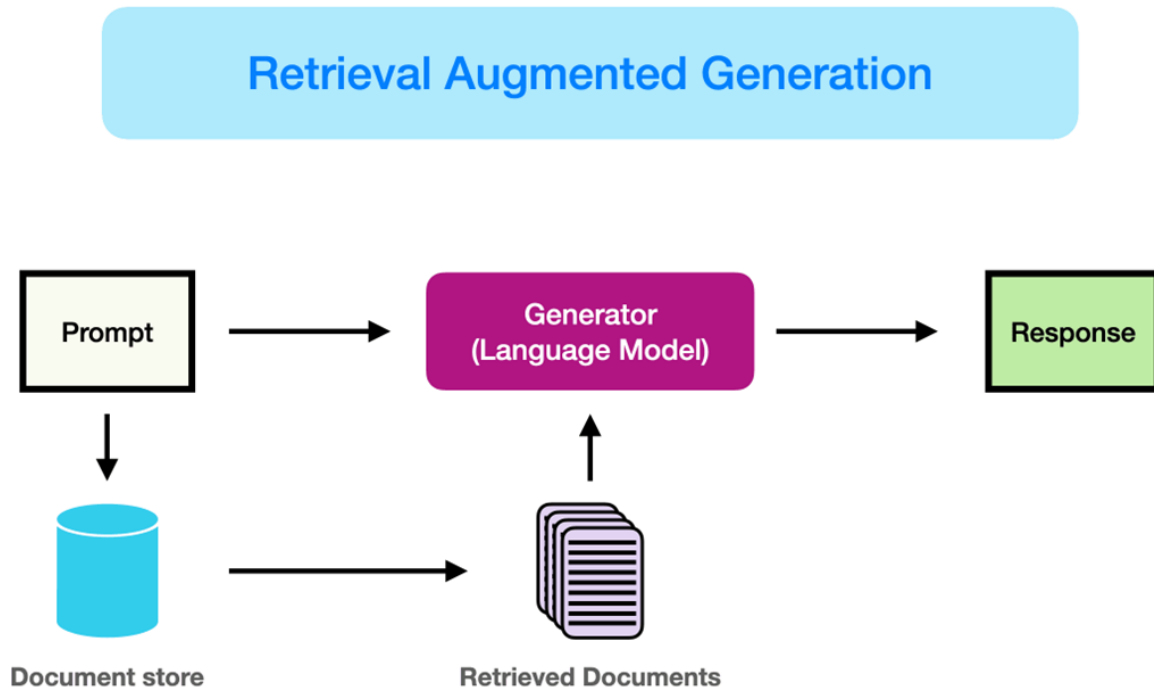


Abbildung 5.2: Architekturdiagramm des RAG-Systems in EduShelf

Der Datenfluss bei einer Benutzeranfrage lässt sich in fünf klar abgegrenzte Schritte gliedern:

1. **Anfrage & Intent-Erkennung:** Der `ChatController` empfängt die Nachricht des Nutzers und leitet sie an den `ChatService` weiter. Dieser übergibt sie zuerst an den `IntentDetectionService`.
2. **Vektorielle Suche (Retrieval):** Basierend auf dem erkannten Intent ruft der `RetrievalService` die semantisch relevantesten `DocumentChunks` aus der PostgreSQL/pgvector-Datenbank ab.
3. **Kontext-Aufbereitung:** Der `PromptGenerationService` kombiniert die Konversationshistorie, die abgerufenen Chunks und die aktuelle Frage zu einem strukturierten Prompt.
4. **LLM-Inferenz:** Über das Microsoft Semantic Kernel SDK wird der Prompt an das lokal laufende Ollama-LLM gesendet.

5. **Antwort & Persistenz:** Die generierte Antwort wird in der Datenbank als `ChatMessage` gespeichert und an den Client zurückgegeben.

5.9.2 Schritt 1: Intent-Erkennung (IntentDetectionService)

Bevor eine semantische Suche stattfindet, muss das System verstehen, *was* der Nutzer möchte. Der `IntentDetectionService` sendet die Benutzeranfrage an das LLM mit einem speziellen System-Prompt, der das Modell anweist, eine strukturierte JSON-Antwort zu liefern:

```
1 var chatHistory = new ChatHistory();
2 chatHistory.AddSystemMessage(
3     _configuration.GetValue<string>("AIService:Prompts:Intent")
4     ?? "Identify the intent.");
5 chatHistory.AddUserMessage(userInput);
6
7 var result = await chatCompletionService.GetChatMessageContentAsync(
8     chatHistory);
9 var cleanedJson = JsonHelper.ExtractJson(result.Content ?? string.Empty);
10 var intent = JsonSerializer.Deserialize<Intent>(cleanedJson,
11     new JsonSerializerOptions { PropertyNameCaseInsensitive = true })
12     ?? new Intent { Type = "question", DocumentName = null };
```

Listing 5.5: Intent-Erkennung via LLM in `IntentDetectionService.cs`

Das `Intent`-Objekt enthält zwei Felder:

- **Type:** Klassifiziert die Anfrage als "question", "summarize", "flashcards" oder "quiz".
- **DocumentName:** Falls der Nutzer explizit ein Dokument nennt (z.B. „Erkläre mir Kapitel 3 aus Physik.pdf“), wird der Dateiname hier extrahiert.

Dieser vorgelagerte Schritt ermöglicht eine fundamentale Weichenstellung im weiteren Ablauf: Statt immer dieselbe RAG-Pipeline auszuführen, wird die Antwort-Strategie dynamisch angepasst.

5.9.3 Schritt 2: Semantische Suche (RetrievalService)

Der `RetrievalService` ist für die Beschaffung der relevanten Dokumentenabschnitte zuständig. Er implementiert dabei eine zweistufige Strategie:

5.9.3.1 Primäre Suche: Name-basiertes Matching

Falls der Intent einen `DocumentName` enthält, sucht der Service direkt nach Chunks, deren übergeordnetes Dokument diesen Namen trägt. Dies erfolgt über einen PostgreSQL-spezifischen, groß-/kleinschreibungsunabhängigen Vergleich (`ILike`):

```

1 var chunks = await _context.DocumentChunks
2   .Include(dc => dc.Document)
3   .Where(dc => dc.Document.UserId == userId &&
4     (EF.Functions.ILike(dc.Document.Title, intent.DocumentName) ||
5     EF.Functions.ILike(dc.Document.Title,
6     documentNameWithoutExtension)))
7   .ToListAsync();

```

Listing 5.6: Dokumenten-spezifische Suche in `RetrievalService.cs`

5.9.3.2 Fallback: Semantische Vektorsuche

Wenn kein Dokumentname angegeben wurde oder keine Treffer gefunden wurden, wird eine semantische Suche über alle Chunks des Nutzers durchgeführt. Die Anfrage wird in einen Embedding-Vektor umgewandelt und mit allen gespeicherten Chunk-Vektoren per L2-Distanz verglichen:

```

1 var promptEmbedding = await _embeddingGenerator.GenerateAsync(userInput);
2 var k = GetDynamicK(userInput); // 5, 10 oder 20 Chunks
3
4 return await _context.DocumentChunks
5   .Include(dc => dc.Document)
6   .Where(dc => dc.Document.UserId == userId)
7   .OrderBy(dc => dc.Embedding.L2Distance(new Vector(promptEmbedding.Vector)))
8   .Take(k)
9   .ToListAsync();

```

Listing 5.7: Embedding-basierte Vektorsuche in `RetrievalService.cs`

Der Wert k (Anzahl der zurückgegebenen Chunks) ist dynamisch: Bei kurzen Anfragen (unter 10 Wörtern) werden $k = 5$, bei mittleren $k = 10$ und bei komplexen Anfragen (über 30 Wörter) $k = 20$ Chunks abgerufen.

5.9.4 Schritt 3: Prompt-Aufbau (PromptGenerationService)

Der PromptGenerationService baut eine vollständige ChatHistory auf, die drei Elemente enthält:

- Eine **System-Instruktion**, die das Verhalten des LLMs grundlegend steuert.
- Die **bisherige Konversationshistorie** der aktuellen Chat-Sitzung (für kontextsensitive Folgefragen).
- Die **aktuelle Benutzeranfrage**, angereichert mit dem abgerufenen Dokumentenkontext.

```

1 // Kontext aus Chunks zusammenstellen und auf Token-Limit kuerzen
2 foreach (var chunk in relevantChunks) {
3     contextText.AppendLine($"[{chunk.Document.Title}] {chunk.Content}");
4 }
5 contextString = _tokenService.Truncate(contextString, contextLengthLimit);
6
7 // Finale Benutzernachricht mit eingebettetem Kontext
8 finalUserMessage.AppendLine($"[Context from Documents]:\n{contextString}\n");
9 ;
10 finalUserMessage.AppendLine($"[User Question]: {userInput}");
11 chatHistory.AddUserMessage(finalUserMessage.ToString());

```

Listing 5.8: Prompt-Konstruktion in PromptGenerationService.cs

Ein zentrales Element ist die **Token-Limitierung** durch den ITokenService. Um das Kontextfenster des LLMs (konfigurierbar, Standard: 4096 Token) nicht zu überschreiten, werden die Chunks vor der Übergabe tokenisiert und präzise abgeschnitten (*Smart Truncation*). Dies verhindert API-Fehler und stellt sicher, dass die semantisch nächsten Chunks priorisiert werden.

5.9.5 Schritt 4: Orchestrierung und Intent-basiertes Branching (ChatService)

Der ChatService fungiert als zentraler Orchestrator und trifft nach der Retrieval-Phase eine **Intent-basierte Fallunterscheidung**. Dies unterscheidet EduShelf von einem einfachen Chatbot:

```

1 var intent = await _intentDetectionService.GetIntentAsync(promptInput);
2 var relevantChunks = await _retrievalService.GetRelevantChunksAsync(
3     promptInput, userId, intent);
4

```

```

5 if (intent.Type == "flashcards") {
6     await _flashcardService.GenerateFlashcardsAsync(
7         new GenerateFlashcardsRequest { Context = context, Count = 5 },
8         userId);
9     responseContent = "I have generated flashcards.";
10 }
11 else if (intent.Type == "quiz") {
12     await _quizService.GenerateQuizAsync(
13         new GenerateQuizRequest { Context = context, Count = 5 }, userId);
14     responseContent = "I have generated a quiz.";
15 }
16 else { // "question" oder "summarize"
17     var chatHistory = _promptGenerationService.BuildChatHistory(
18         chatSession, relevantChunks, promptInput);
19     var result = await chatCompletionService.GetChatMessageContentAsync(
20         chatHistory);
21     responseContent = result.Content ?? "Sorry, no response generated.";
22 }

```

Listing 5.9: Intent-basiertes Branching in ChatService.cs

Dieses Design ermöglicht es, über eine einzige Chat-Schnittstelle drei fundamentale Lernfunktionen anzubieten: konversationelles Frage-Antwort, automatische Flashcard-Generierung und automatische Quiz-Erstellung – alles gesteuert durch die Intention hinter der natürlichsprachlichen Eingabe des Nutzers.

5.9.6 Zusammenfassung: Retrieval-Augmented Generation

Die Implementierung des RAG-Systems stellt sicher, dass das LLM stets auf den spezifischen Kontext der hochgeladenen Dokumente des Nutzers antwortet. Die wichtigsten Designprinzipien sind:

- **Datenprivacy:** Durch lokale Ollama-Inferenz verlassen keine Dokumenteninhalte den Server.
- **Präzision:** Durch semantische Vektorsuche und optionales Name-Matching werden nur die relevantesten Chunks an das LLM übergeben.
- **Robustheit:** Token-basierte Kontextlimitierung verhindert LLM-Abstürze durch überlange Prompts.
- **Erweiterbarkeit:** Das Intent-Branching erlaubt es, neue KI-gestützte Features durch Hinzufügen weiterer Intent-Typen zu integrieren, ohne die Kernpipeline zu ändern.

5.10 KI-gestützte Lerninhaltsgenerierung

Neben der RAG-basierten Chat-Funktionalität bietet EduShelf zwei weitere KI-Features: die automatische Generierung von **Lernkarten (Flashcards)** und **Multiple-Choice-Quizzes**. Beide Features nutzen denselben Ollama-LLM-Stack, folgen aber einem eigenständigen, direkteren Prompt-Workflow, da hier keine Vektorsuche benötigt wird – der Nutzer liefert den Kontext direkt über Dokumenten-IDs oder RAG-abgerufene Chunks.

5.10.1 Generierungspipeline (5 Schritte)

Beide Services (`FlashcardService`, `QuizService`) folgen derselben fünfstufigen Pipeline:

1. **Kontextbeschaffung:** Der Service akzeptiert drei Eingabequellen: einen direkten `Context-String` (aus der RAG-Pipeline), eine einzelne `DocumentId` oder eine Liste von `DocumentIds`. Im letzteren Fall werden die Textinhalte aller Dokumente via `GetDocumentContentAsync` abgerufen und mit einem Trennzeichen (`\n\n__NEXT_DOCUMENT__\n\n`) zusammengefügt.
2. **Kontext-Truncation:** Um das Kontextfenster des LLMs nicht zu sprengen, wird der Dokumenteninhalt auf maximal 12.000 Zeichen (Flashcards) bzw. 15.000 Zeichen (Quizze) begrenzt.
3. **Prompt-Vorbereitung:** Der System-Prompt wird aus `appsettings.json` geladen (`AIService.Prompts.Flashcard` bzw. `:Quiz`). Die Variable `{Count}` wird durch die gewünschte Anzahl ersetzt.
4. **LLM-Inferenz:** Das Modell wird mit dem vorbereiteten `ChatHistory`-Objekt aufgerufen.
5. **JSON-Parsing und Persistenz:** Die Antwort wird geparkt und in der Datenbank gespeichert.

5.10.2 Flashcard-Generierung (FlashcardService)

Der LLM wird angewiesen, eine JSON-Liste von Objekten mit den Feldern **Question** und **Answer** zurückzugeben. Das folgende Snippet zeigt den Prompt-Aufbau und die anschließende Verarbeitung:

```

1 // Prompt zusammenbauen und KI aufrufen
2 var chatHistory = new ChatHistory();
3 chatHistory.AddSystemMessage(
4     promptTemplate.Replace("{Count}", request.Count.ToString()));
5 chatHistory.AddUserMessage(
6     $"Text to analyze:\n\n{documentContent}\n\n" +
7     "IMPORTANT: Output strictly valid JSON. Start with [."");
8
9 var result = await chatCompletionService
10     .GetChatMessageContentAsync(chatHistory);
11
12 // JSON bereinigen und deserialisieren
13 var cleanedJson = JsonHelper.ExtractJson(result.Content);
14 var flashcards = JsonSerializer.Deserialize<List<GeneratedFlashcardJson>>(
15     cleanedJson,
16     new JsonSerializerOptions { PropertyNameCaseInsensitive = true });

```

Listing 5.10: KI-Aufruf und JSON-Parsing in FlashcardService.cs

Nach dem Parsing werden die generierten Karten automatisch mit einem Tag versehen, der den Dokumenttitel trägt, und als **Flashcard**-Entitäten in der Datenbank persistiert.

5.10.3 Quiz-Generierung (QuizService)

Die Quiz-Generierung ist komplexer, da das LLM ein verschachteltes JSON-Objekt mit **Title**, **Questions** und pro Frage mehrere **Answers** (jeweils mit **IsCorrect**-Flag) produzieren muss. Da LLMs trotz strikter Anweisung gelegentlich vom erwarteten Schema abweichen (z.B. alternative Feldnamen wie **options** statt **Answers**), implementiert der **QuizService** eine **zweistufige Parsing-Strategie**:

```

1 var jsonNode = JsonNode.Parse(cleanedJson);
2
3 // Stufe 1: Wrapper-Handling (falls LLM { "quiz": {...} } liefert)
4 if (jsonNode["quiz"] != null)
5     jsonNode = jsonNode["quiz"];
6
7 // Stufe 2a: Standard-Deserialisierung versuchen
8 var result = jsonNode.Deserialize<GeneratedQuizJson>(options);
9
10 // Stufe 2b: Fallback -- Manuelles Parsing bei alternativen Schemas
11 if (result == null || !result.Questions.Any())
12 {

```

```
13 // Felder wie "text"/"Text", "options"/"answers", "answer" (string)
14 // werden manuell ausgelesen und normalisiert.
15 result = ManuallyParseQuizJson(jsonNode);
16 }
```

Listing 5.11: Robustes JSON-Parsing in QuizService.cs

Die manuelle Fallback-Stufe erkennt dabei sowohl Antwortoptions als einfache Strings (`[Paris", London", ...]`) als auch als Objekte (`[{text: ..., isCorrect: ...}]`). Dies macht die Quiz-Generierung robust gegenüber variierenden LLM-Antwortformaten.

Nach erfolgreichem Parsing wird das Quiz als `Quiz`-Entität mit allen `Question`- und `Answer`-Objekten in einer einzigen EF Core-Transaktion gespeichert.

5.11 Infrastruktur & Deployment

Das System ist vollständig containerisiert ("Dockerized"), um eine konsistente Umgebung von der Entwicklung bis zur Produktion zu gewährleisten.

5.11.1 Docker-Komposition

Die `docker-compose.yml` definiert das Zusammenspiel der Dienste und deren spezifische Konfiguration für die Produktionsumgebung.

5.11.1.1 Datenbank (PostgreSQL mit pgvector)

Für die persistente Speicherung und Vektorsuche wird ein spezialisiertes Image verwendet:

- **Image:** `pgvector/pgvector:pg16` - Dies ist essentiell, da das Standard-Postgres-Image die `vector`-Erweiterung nicht enthält.
- **Ports:** Das Mapping `5433:5432` vermeidet Konflikte mit lokalen Postgres-Instanzen auf dem Host.
- **Healthcheck:** Ein integrierter Healthcheck (`pg_isready`) stellt sicher, dass abhängige Container (wie die API) erst starten, wenn die Datenbank vollständig hochgefahren ist.

- **Persistenz:** Das Volume `postgres_data` sichert die relationalen Daten und Vektor-Indizes über Container-Neustarts hinweg.

5.11.1.2 KI-Inferenz (Ollama)

Der Ollama-Container ist für die lokale Ausführung der LLMs zuständig und benötigt direkten Hardware-Zugriff für performante Inferenz:

- **GPU-Passthrough:** Über die `deploy.resources`-Sektion wird der Nvidia-Treiber durchgereicht:

```
reservations:
  devices:
    - driver: nvidia
      count: 1
      capabilities: [gpu]
```

Dies ermöglicht die Nutzung von CUDA-Kernen, was die Antwortzeit (Time-to-First-Token) drastisch reduziert (von Sekunden auf Millisekunden).

- **Persistenz:** Das Volume `ollama_data` speichert die heruntergeladenen Modelle (z.B. Llama 3) im `/root/.ollama` Verzeichnis, um erneute Downloads bei Container-Updates zu vermeiden.
- **Keep-Alive:** Die Umgebungsvariable `OLLAMA_KEEP_ALIVE` verhindert, dass das Modell nach kurzer Inaktivität aus dem VRAM entladen wird, was die Latenz bei aufeinanderfolgenden Anfragen minimiert.

5.11.1.3 Object Storage (MinIO)

MinIO dient als skalierbarer, S3-kompatibler Dateispeicher.

- **Ports:**
 - 9000: API-Port für die Kommunikation mit dem Backend (S3-Protokoll).
 - 9001: Web-Interface (Console) zur manuellen Verwaltung von Buckets und Dateien.

- **Command:** Der Startbefehl `server /data -console-address :9001` aktiviert das Web-UI explizit.
- **Volumes:** `minio_data` speichert die physischen Dateien (PDFs, Bilder) persistent.

5.11.2 Dockerfile

Das Dockerfile der API nutzt Multi-Stage Builds, um das Image klein zu halten:

1. **Build Stage:** Nutzt das vollständige SDK Image (`mcr.microsoft.com/dotnet/sdk:8.0`), um den Code zu kompilieren und Abhängigkeiten (NuGet Pakete) zu laden.
2. **Runtime Stage:** Nutzt das schlanke Runtime Image (`mcr.microsoft.com/dotnet/aspnet:8.0`), welches nur die zum Ausführen nötigen Komponenten enthält. Das kompilierte Artefakt aus Stage 1 wird hierhin kopiert.

Dies reduziert die Image-Größe drastisch und verbessert die Sicherheit, da keine Compiler-Tools im Produktions-Container vorhanden sind.

5.12 Resilienz und Stabilität

In verteilten Systemen, insbesondere bei der Anbindung von KI-Diensten, sind temporäre Ausfälle unvermeidbar. Um die Robustheit der Anwendung zu gewährleisten, wurde die Bibliothek Polly integriert.

5.12.1 Retry-Policies & Circuit Breaker

In der `ServiceCollectionExtensions.cs` werden resiliente HTTP-Clients konfiguriert:

1. **Retry Policy:** Bei transienten Fehlern (z.B. Netzwerk-Glitch, 503 Service Unavailable) wird der Request bis zu 3-mal wiederholt, mit einem exponentiellen Backoff (2^n Sekunden).
2. **Circuit Breaker:** Wenn 5 Fehler in Folge auftreten, "öffnet" sich der Stromkreis für 30 Sekunden. Alle weiteren Anfragen werden sofort abgeblockt, um das überlastete Subsystem (z.B. Ollama) zu schonen.

```

1 var retryPolicy = HttpPolicyExtensions
2   .HandleTransientHttpError()
3   .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2,
4     retryAttempt)));
5 var circuitBreakerPolicy = HttpPolicyExtensions
6   .HandleTransientHttpError()
7   .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30));

```

Listing 5.12: Polly-Konfiguration

5.13 Herausforderungen und Lösungen

Während der Backend-Entwicklung mussten mehrere technische Hürden genommen werden.

5.13.1 Blockierende Operationen bei großen Dateien

Lösung: Einführung einer BackgroundJobQueue. Der Upload-Endpoint nimmt die Datei entgegen, speichert sie in MinIO und reiht nur eine "Job-ID" in die Queue ein, bevor er sofort 202 Accepted zurückgibt. Ein Hintergrunddienst ("Worker") arbeitet diese Queue asynchron ab.

```

1 protected override async Task ExecuteAsync(CancellationToken stoppingToken)
2 {
3     while (!stoppingToken.IsCancellationRequested)
4     {
5         var workItem = await _queue.DequeueAsync(stoppingToken); //
6         // Blockiert bis Arbeit da ist
7         try
8         {
9             await workItem(stoppingToken); // Fuehrt den eigentlichen
10            // Indexing-Task aus
11        }
12        catch (Exception ex)
13        {
14            _logger.LogError(ex, "Error occurred executing background work
15            item.");
16        }
17    }
18 }

```

Listing 5.13: Verarbeitungsschleife im BackgroundJobService

5.13.2 LLM Kontext-Fenster-Limits

Problem: GPT-4 und andere Modelle haben ein festes Limit an Tokens (z.B. 4096 oder 8192). Ein blindes Einfügen von Dokumenten würde zu Abstürzen führen. **Lösung:** Implementierung des `TokenService` mit *Tiktoken*. Bevor ein Prompt generiert wird, werden die Tokens der abgerufenen Chunks gezählt. Ist das Limit erreicht, werden die am wenigsten relevanten Chunks verworfen ("SSmart Truncation").

5.13.3 Datenkonsistenz bei Löschoperationen

Problem: Wenn ein User gelöscht wird, dürfen dessen Dokumente und Vektoren nicht als "Datenmüllin der Datenbank verbleiben. **Lösung:** Nutzung von `onDelete(DeleteBehavior.Cascade)` in EF Core. Da auch die Vektoren (`DocumentChunk`) über Foreign Keys hart an das `Document` und dieses an den `User` gekoppelt ist, löscht ein einziger SQL-Befehl (`DELETE FROM Users WHERE Id=...`) kaskadierend den gesamten assoziierten Datenbaum.

5.14 Qualitätssicherung und Tests

Die Stabilität des Backends wird durch automatisierte Tests gewährleistet.

5.14.1 Unit Tests (xUnit)

Die Geschäftslogik (`Services`) wird isoliert getestet. Externe Abhängigkeiten wie Datenbank oder File-System werden mittels `Moq` gemockt. Beispiel: Ein Test für den `FlashcardService` prüft, ob die richtigen DTOs zurückgegeben werden, ohne dass dabei eine echte KI angefragt wird (Mocking des `IChatCompletionService`).

5.14.2 Integration Tests

Für Controller und die Datenbankinteraktion werden Integrationstests verwendet. Hierbei wird mittels `WebApplicationFactory` ein echter Test-Server im Speicher hochgefahren, der gegen eine In-Memory-Datenbank oder eine Docker-Test-Instanz läuft. Dies stellt sicher, dass die Middleware-Pipeline, das Routing und das Entity Framework korrekt zusammenspielen.

5.15 Fazit

Das Backend von EduShelf stellt ein robustes, skalierbares Fundament für die Lernplattform dar. Durch die Kombination von bewährten Mustern (Layered Architecture, DI) mit modernsten KI-Technologien (Vektor-Datenbanken, RAG) konnte eine leistungsfähige Lösung für das Dokumentenmanagement und interaktive Lernen geschaffen werden.

Zeiterfassung



Abbildung 5.3: AI und iOS Zeitaufwand

Gesamt: 263,17h

Freizeit: 228,48h

Abbildungsverzeichnis

4.1	AI und iOS Zeitaufwand	39
5.1	Entity Relationship Diagram des EduShelf Backends	48
5.2	Architekturdiagramm des RAG-Systems in EduShelf	55
5.3	AI und iOS Zeitaufwand	68

Code-Verweise

4.1	Auszug aus der Vite-Konfiguration	27
4.2	Routing-Struktur in App.jsx	28
4.3	Generische GET-Methode im API-Service	29
4.4	CSS für den Flip-Effekt	31
4.5	Theme-Switching mittels CSS-Variablen	33
4.6	Initialer State im QuizModal	34
4.7	Update-Logik für verschachtelte Antworten	34
4.8	Nutzung eines Sets fuer UI-State	35
4.9	Asynchrones Laden von Detaildaten	36
4.10	Dockerfile für das Frontend	37
5.1	Auszug aus der Middleware-Konfiguration	44
5.2	Konfigurationsstruktur der appsettings.json	45
5.3	Implementierung der Satz-Segmentierung in TextChunker.cs	52
5.4	Verifizierung im AuthService	53
5.5	Intent-Erkennung via LLM in IntentDetectionService.cs	56
5.6	Dokumenten-spezifische Suche in RetrievalService.cs	57
5.7	Embedding-basierte Vektorsuche in RetrievalService.cs	57
5.8	Prompt-Konstruktion in PromptGenerationService.cs	58
5.9	Intent-basiertes Branching in ChatService.cs	58
5.10	KI-Aufruf und JSON-Parsing in FlashcardService.cs	61
5.11	Robustes JSON-Parsing in QuizService.cs	61
5.12	Polly-Konfiguration	65
5.13	Verarbeitungsschleife im BackgroundJobService	65